AI Applications in Research
Concepts and Practice with Claude Code

A structured concept summary

Compiled 24 September 2026

Abstract

This document summarises the main concepts behind the use of artificial intelligence in empirical research, with a focus on agentic coding assistants and, in particular, **Claude Code**. It draws on the ongoing public documentation of these workflows by economist Scott Cunningham ("Scott's Mixtape" Substack) and related public discussion. The goal is not a software manual but a conceptual map: what changes when a capable coding agent becomes a routine collaborator in the research pipeline, which practices make that collaboration trustworthy, and which risks the practice introduces.

1 From code assistants to research agents

The recent shift in AI for research is from *autocomplete*-style assistance to *agentic* assistance. An agentic coding tool does not merely suggest the next line; given a goal, it plans, edits files, runs code, reads the output, and iterates across many turns until a completion condition is met. Claude Code is a terminal-based agent of this kind: it operates directly on a project directory, executes commands, and reasons over the results.

For the empirical researcher this reframes the unit of work. Instead of writing every line of a regression pipeline, the researcher *specifies intent* and *audits output*. The comparative advantage shifts from typing code toward framing questions, designing checks, and judging whether results are credible.

Core idea. An AI research agent turns the researcher into an *editor and referee* of a pipeline rather than only its author. Value concentrates in specification, verification, and interpretation.

2 Principal applications in the research pipeline

2.1 Replication and cross-language auditing

A recurring application is using an agent to **replicate a pipeline from scratch** in several languages and packages, then checking whether the non-random intermediate results agree. Cunningham illustrates this with difference-in-differences estimated across Python, R, and Stata using multiple packages (e.g. Stata's `csdid`/`csdid2`, R's `did`, and Python implementations), focusing on estimators such as Callaway and Sant'Anna. Agreement across independent implementations is strong evidence that a result is not an artefact of one package's defaults.

2.2 Multi-agent auditing and the "Referee 2" protocol

Perhaps the most important methodological contribution is the principle that **you cannot grade your own homework**. Asking the same agent that wrote the code to review it is weak: it shares the author's blind spots. The "Referee 2" protocol instead runs a structured audit in a *fresh* session, by an agent instance that has never seen the work, producing a formal referee

report and a revise-and-resubmit loop. Independence between the builder and the auditor is what makes the check meaningful.

2.3 Reproducing published findings

Benchmarks reported in 2026 suggest AI coding agents can reproduce social science findings at high rates—on the order of ∼93% accuracy at the task level and ∼78% at the paper level in one study—substantially above earlier agents. This makes automated *reproducibility auditing* of the existing literature a realistic activity rather than an aspiration.

2.4 Communication artefacts: "beautiful decks"

A second class of application is turning analysis into communication. Agents can extract the tacit rhetoric of good presentations from their training data and assemble "beautiful decks" (e.g. LaTeX/Beamer slides) that explain a paper or a result. Cunningham highlights this as a likely "killer app" for non-programmers, because it lowers the cost of high-quality exposition close to zero.

2.5 Teaching, lecture preparation, and legacy code

Related uses include generating lecture slides and materials, and opening a years-old project directory and having the agent reconstruct, document, and modernise the workflow—resurrecting code whose context the author has largely forgotten.

3 Making the collaboration trustworthy

The practices that distinguish reliable AI-assisted research from careless use are procedural. The following recur across the material:

- **Independence of verification.** Audit in a clean context with an agent that did not write the code (the "Referee 2" idea).
- **Cross-implementation agreement.** Re-derive key numbers in more than one language/-package and require they match.
- **Explicit checklists.** Applied econometrics benefits from a detailed, standing checklist the agent must satisfy—defaults, sample construction, clustering, inference—rather than trusting narrative.
- **A workflow "harness."** Reusable skills, quality gates, and adversarial QA steps package good practice so it is applied every time, not re-invented per project.
- **Beauty and story as signals.** Cunningham argues that clarity, aesthetics, and narrative in the output are not decoration but part of the harness: work that is legible is easier to check and to trust.

> **Design principle.** Build *independence* and *redundancy* into the pipeline. Trust comes from a result surviving separate, adversarial checks, not from the fluency of a single agent's answer.

4 The economics of AI-assisted research

A distinctive theme is the **falling marginal cost of cognitive output**. Once a task can be handed to an agent, the private marginal cost of producing an additional unit of "cognitive

output"—a table of coefficients, a poem, or an entire worked manuscript—approaches epsilon. Cunningham frames this with an isoquant for producing a unit of such output.

Two implications follow. First, output volume can explode; with it, **aggregate error** can rise even as each task gets cheaper, so verification becomes *more* important, not less. Second, private and social marginal costs diverge: even if providers price usage at private marginal cost, that need not internalise broader costs (including long-run and existential risk). Cheapness of production is therefore not the same as cheapness to society.

5 Risks, limitations, and open problems

- **New problems alongside solved ones.** Adopting an agent can solve old bottlenecks while creating new ones, even without saving time— e.g. effort shifts to specification and review.
- **Memory and "amnesia."** Agents lose context across sessions; important reasoning and decisions must be captured durably or they vanish. Persisting the agent's stated *reasons* is itself a task.
- **Plausible-but-wrong output.** Fluent results can be subtly incorrect; independent auditing and checklists are the guardrails.
- **Scholarly-communication strain.** Cheap, high-volume output pressures peer review, disclosure norms, and journal practices (e.g. debates over AI-disclosure and the economics of publishing).
- **Concentration of judgement.** As typing is automated, the scarce input becomes expert judgement—the "returns to expertise" rise.

6 Case study: documenting the practice in the open

Scott Cunningham's "Claude Code" series (running since roughly mid-November 2025 into the mid-50s of instalments by autumn 2026) is a running, public laboratory notebook of these ideas. Rather than abstract claims, it shows live video and written walkthroughs of real empirical work—replication audits, deck generation, workflow-harness design, and reflections on the economics and epistemics of the practice. Supporting tooling is shared publicly (e.g. the *Mixtape Tools* project and community workflow templates), which lets others adopt the "Referee 2" audits and harness patterns directly.

Takeaway. The most transferable output of this work is not a specific script but a *method*: specify intent, produce redundantly, verify independently, and document reasons—so that cheap cognitive output remains credible.

7 Selected references and resources

- Scott Cunningham, *Scott's Mixtape* Substack — https://causalinf.substack.com/
- "Claude Code" series (index) — https://causalinf.substack.com/s/claude-code
- *Claude Code 56: Falling marginal costs of production, output, and aggregate errors* — https://causalinf.substack.com/p/claude-code-56-falling-marginal-costs
- *Claude Code 55: Beauty and story are key to my workflow harness* — https://causalinf.substack.com/p/claude-code-55-beauty-is-key-to-the

- *Claude Code 53: Applied econometrics will require a detailed checklist* — https://causalinf.substack.com/p/claude-code-53-applied-econometrics
- *Claude Code 24: Multiple Agents Auditing Your Diff-in-Diff Code* — https://causalinf.substack.com/p/claude-code-24-multiple-agents-auditing
- MixtapeTools (public tooling) — https://github.com/scunning1975/MixtapeTools
- "AI Coding Agents Can Reproduce Social Science Findings" (arXiv) — https://arxiv.org/abs/2606.11447
- "CausalVerify: An Execution-Grounded Benchmark for LLM Causal Inference Workflows" (arXiv) — https://arxiv.org/abs/2609.07944

Note: Direct fetching of the Substack was unavailable in the compilation environment; the substance above was assembled from search-indexed summaries of the cited posts and may not capture every nuance of the originals.